

Universidad Autónoma de San Luis Potosí

FACULTAD DE INGENIERÍA

CARRERA:

Ingeniería en Sistemas Inteligentes

ASIGNATURA:

Lenguajes de programación

ELABORADO POR:

369664 Antonio Jiménez José Emilio
388531 Alvarez Puente Alan Antonio

DOCENTE:

CICLO ESCOLAR:

2025 - 2026

San Luis Potosí, S.L.P.

19 de mayo de 2026

Índice

1. Estructuras de Datos y Objetos	3
Entidad	3
Atributos	4
CDiccionario	5
2. Funciones Entidad	7
Menu Entidades	7
Captura Entidad	7
Alta Entidad	7
Busca Entidad	7
Get Cabecera Entidades	8
Lee Entidad	8
Escribe Cabecera Entidades	8
Escribe Entidad	8
Inserta Entidad	8
Consulta Entidades	9
Baja Entidad	9
Elimina Entidad	9
Reescribe Entidad	9
Modifica Entidad	9
3. Funciones Atributo	10
Menu Atributos	10
Captura Atributo	10
Alta Atributo	10
Buscar Atributo	10
Lee Atributo	10
Escribe Atributo	10
Inserta Atributo	11
Consultar Atributo	11
Baja Atributo	11
Elimina Atributo	12
Reescribe Atributo	12
Modifica Atributo	12
4. Diagramas	13
5. Archivo	14
6. Clave primaria	15
7. Conclusiones	16

Referencias

17

1 Estructuras de Datos y Objetos

Descripción de la Estructura Entidad:

La estructura es el molde inicial de nuestro Diccionario en formato de base de datos de acceso aleatorio. Se establece un alias `cadena` para asegurar que cada registro tenga el mismo tamaño.

- **cadena nombre:** Identificador único de cada registro de entidad (máximo 30 caracteres).
- **long atr:** Puntero tipo `long` que conecta la entidad con su lista de atributos.
- **long data:** Dirección física en el archivo donde se almacenan los registros de datos reales.
- **long sig:** Enlace a la siguiente entidad, permitiendo una estructura de lista ligada.

```
1 #ifndef ENTIDAD_H_INCLUDED
2 #define ENTIDAD_H_INCLUDED
3 #include <iostream>
4 #include <cstdio>
5
6 typedef char cadena[30];
7 struct Entidad
8 {
9     cadena nombre;
10    long atr;
11    long data;
12    long sig;
13 };
14 #endif
```

Estructura 1: Estructura de Datos de Entidad

Descripción de la Estructura Atributo:

Esta estructura es el molde para las columnas de nuestro Diccionario. Al igual que en la entidad, se usa el alias `cadena` para mantener el tamaño fijo en el archivo binario y evitar que los registros se desfasen. En esta estructura observamos las siguientes declaraciones:

- **cadena nombre:** Identificador del atributo dentro de la entidad.
- **unsigned int tipo:** Código numérico que define el tipo de dato (ej. entero, flotante, cadena).
- **unsigned int tamano:** Longitud en bytes que ocupará el valor del atributo en el registro.
- **char iskp:** Bandera (flag) que indica si el atributo funciona como llave primaria (Primary Key).
- **long sig:** Puntero lógico (offset) que enlaza con el siguiente atributo de la entidad.
- **cadena descripcion:** Información adicional o etiqueta descriptiva del campo.
- **char null:** Indicador booleano que define si el campo permite valores nulos.

```
1 #ifndef ATRIBUTO_H
2 #define ATRIBUTO_H
3
4 typedef char cadena [30];
5
6 typedef struct{
7     cadena nombre;
8     unsigned int tipo;
9     unsigned int tamano;
10    char iskp;
11    long sig;
12    cadena descripcion;
13    char null;
14 }Atributo;
15
16 #endif
```

Estructura 2: Estructura de Datos de Atributos

Descripción del Objeto CDiccionario:

La clase CDiccionario funciona como el administrador principal del sistema. En su parte privada, contiene las variables necesarias para controlar el flujo del archivo binario y saber exactamente en qué parte del disco estamos parados mientras el usuario navega. En esta sección observamos las siguientes declaraciones:

- **FILE *arch:** Es el puntero al archivo físico donde se almacenan todas las entidades, atributos y datos.
- **Entidad activa:** Representa la entidad que el usuario está manipulando o consultando en ese momento.
- **long dir:** Guarda la dirección física (offset) actual dentro del archivo para las operaciones de lectura/escritura.
- **int nAtributos:** Es un contador que lleva el registro de cuántos atributos tiene la entidad seleccionada.
- **long tambloque:** Almacena el tamaño del bloque activo para gestionar correctamente el espacio en el archivo.
- **char nombreArchivo[50]:** Es el arreglo donde se guarda el nombre del archivo que el usuario decide abrir o crear.

```
1 #ifndef CDICCIONARIO_H_INCLUDED
2 #define CDICCIONARIO_H_INCLUDED
3
4 #include <cstdio>
5 #include "Atributo.h"
6 #include "Entidad.h"
7
8 class CDiccionario
9 {
10 private:
11     FILE *arch;
12     Entidad activa;
13     long dir;
14     int nAtributos;
15     long tambloque;
16
17     char nombreArchivo[50];
18 public:
19     //constructor
20     CDiccionario();
21
22     //Entidades
23     void Menu_Entidades();
24     Entidad CapturaEntidad();
25     void altaEntidad();
26     long buscaEntidad(char nombre[30]);
27     long getcabeceraEntidades();
```

```
28     Entidad leeEntidad(long dir);
29     void EscribeCabEntidades(long cab);
30     long escribeEntidad(Entidad ant);
31     void insertaEntidad(Entidad nvo, long dir);
32     void ConsultaEntidades();
33     void BajaEntidad();
34     long EliminaEntidad(char nombre[30]);
35     void reescribeEntidad(long dir, Entidad ant);
36     void modificaEntidad();
37
38     //Atributos
39     void Menu_Atributos();
40     void consultarAtributo();
41     long eliminaAtributo(char cad[30]);
42     void modificaAtributo();
43     void AltaAtributo();
44     Atributo CapturaAt();
45     void insertaAtributo(Atributo nvo, long dir);
46     long BuscarAtributo(char *atr);
47     Atributo leeAtributo(long dir);
48     long EscribeAtributo(Atributo ant);
49     void reescribeAtributo(long dir, Atributo atr);
50     void BajaAtributo();
51
52     //Datos
53     void Menu_Datos();
54
55     //Principal
56     void menu();
57     void Nuevo();
58     void abrir();
59
60 };
61
62 #endif
```

Estructura 3: Estructura de Datos CDiccionario

2 Funciones

Entidad

Menu Entidades

Prototipo: `void CDiccionario::Menu_Entidades();`

La función `menu_Entidades()` muestra un menú interactivo por consola para administrar las funciones relacionadas a Entidades. Funciona por medio de un ciclo `do-while` y un `switch-case` que lee la opción del usuario, para ejecutar acciones como dar de alta, consultar, eliminar o modificar entidades, por consiguiente permitir la navegación hacia la edición de sus atributos y datos.

Captura Entidad

Prototipo: `Entidad CDiccionario::CapturaEntidad();`

La función `CapturaEntidad()` es la plantilla principal para crear registros, se usa una variable de tipo `Entidad`, primero se le pide al usuario el nombre de la entidad, después se inicializa las variables `nueva.atr` conexión con la lista de atributos", `nueva.data` "la dirección real de donde se encuentran los datos de los bloques, y `nueva.sig` que es la dirección a la siguiente entidad, con esto se crea un registro nuevo con sus listas vacías.

Alta Entidad

Prototipo: `void CDiccionario::altaEntidad();`

Esta función es la encargada de verificar que todo esté en condiciones para que se cree una entidad, primero manda a llamar `CapturaEntidad` para crear un registro temporal, seguido de esto viene la verificación con la función `buscaEntidad` para verificar que no haya una entidad con ese mismo nombre, en caso que no exista usa la función `escribeEntidad(nueva)` y guarda la dirección en un `long dir` para finalmente usar la función `insertaEntidad` para escribir en el archivo.

Busca Entidad

Prototipo: `long CDiccionario::buscaEntidad(char nombre[1]);`

La función `long CDiccionario::buscaEntidad(char nombre[])` verifica si se encuentra la entidad, en caso de que sí la encuentre dentro del recorrido `while(cab != -1)` regresa la dirección en donde se encuentra, en caso contrario regresa un valor de -1, dentro del ciclo hace un string compare de la posición donde se encuentra y el nombre que se le haya pasado.

Get Cabecera Entidades

Prototipo: `long CDiccionario::getcabeceraEntidades();`

Esta función es la encargada de obtener la dirección dentro del archivo de donde inicia nuestra lista de entidades, empieza usando un `long dir`, para después con `fseek` dirigirse al inicio de nuestro archivo, con un `fread` extraemos la dirección y la guardamos en la variable `long dir`, sabemos que esa es la cabecera ya que todo el programa está construido para que en el inicio del archivo se encuentra nuestra cabecera de la lista de entidades.

Lee Entidad

Prototipo: `Entidad CDiccionario::leeEntidad(long dir);`

Se encarga de extraer la entidad del archivo, esta función llega con una dirección, esa dirección a partir del `buscaEntidad`, es usada para moverse dentro del archivo, hacer una copia y regresar esa copia para su posterior modificación o consulta.

Escribe Cabecera Entidades

Prototipo: `void CDiccionario::EscribeCabEntidades(long cab);`

La función empieza con un paso de parámetro de la cabecera, dentro de esta se mueve al inicio del archivo y en ese inicio escribe lo que haya tenido el parámetro de nombre `cab`.

Escribe Entidad

Prototipo: `long CDiccionario::escribeEntidad(Entidad ant);`

Esta función se encarga de registrar físicamente una nueva entidad en el archivo, donde se le pasa una copia de la entidad, la función usa un `fseek` con `SEEK_END` para ir hasta el final del último registro del archivo y en ese espacio de memoria usa un `fwrite` para guardarlo en el archivo.

Inserta Entidad

Prototipo: `void CDiccionario::insertaEntidad(Entidad nvo, long dir);`

Esta función se encarga de enlazar secuencialmente de manera lógica una nueva entidad verificando tres casos: si el archivo está vacío, si va al inicio apoyándose de un `strcmp`, o si busca su lugar usando un ciclo `while`. En esencia, como `escribeEntidad` siempre guarda físicamente los datos al final, esta función solo reconecta las variables `nvo.sig` para que apunten de manera correcta y mantengan el orden.

Consulta Entidades

Prototipo: `void CDiccionario::ConsultaEntidades();`

Esta función es la encargada de imprimir en consola las entidades que se encuentran dentro de nuestro archivo. Para ello, hace uso de la función `getcabeceraEntidades` para ver dónde inicia la cabecera de la entidad; subsecuentemente hace uso de un ciclo `while (cab != -1)`, el cual va imprimiendo cada entidad con la dirección de sus componentes.

Baja Entidad

Prototipo: `void CDiccionario::BajaEntidad();` Su única función es preguntar al usuario por el nombre de la entidad, este nombre se lo pasa a la función `EliminaEntidad` en caso de que no exista muestra en consola un aviso al usuario, por el contrario si existe, muestra un aviso que fue eliminado.

Elimina Entidad

Prototipo: `long CDiccionario::EliminaEntidad(char nombre[]);` Esta función empieza haciendo una verificación, si el archivo está vacío regresa un -1, para después verificar si el lugar actual donde está parado es la entidad a borrar, en caso que no hace un recorrido donde hace uso de `while & strcmp`, para encontrar donde está, uso una variable anterior para poder hacer la desconexión de la entidad, no borra nada dentro del archivo solo desconecta y conecta con su inmediato siguiente, lo actualiza en el archivo con la función `reescribeEntidad`.

Reescribe Entidad

Prototipo: `void CDiccionario::reescribeEntidad(long dir, Entidad ant);` Esta función se encarga de actualizar los datos de un registro ya existente en el archivo binario. Recibe una dirección y una estructura con la información modificada, usa `fseek` con `SEEK_SET` para posicionarse exactamente en la dirección proporcionada, y `fwrite` para sobrescribir el bloque de datos. Es indispensable para actualizar los apuntadores lógicos durante operaciones de inserción y baja.

Modifica Entidad

Prototipo: `void CDiccionario::modificaEntidad();` Esta función permite actualizar los datos de un registro. Tras verificar que exista y capturar la nueva información, hace uso de una estrategia segura para mantener el orden alfabético de la lista: elimina lógicamente el registro antiguo con `EliminaEntidad` y después guarda y reubica la entidad actualizada utilizando `escribeEntidad` e `insertaEntidad`.

3 Funciones

Atributo

Menu Atributos

Prototipo: void CDiccionario::Menu_Atributos();

La función Menu_Atributos() muestra un menú interactivo por consola para administrar las funciones relacionadas a Atributos. Funciona por medio de un ciclo do-while y un switch-case que lee la opción del usuario, para ejecutar acciones como dar de alta, consultar, eliminar o modificar atributos, por consiguiente permitir la navegación hacia la edición de sus componentes.

Captura Atributo

Prototipo: Atributo CDiccionario::CapturaAt();

La función CapturaAt() es la plantilla principal para crear registros, se usa una variable de tipo Atributo, primero se le pide al usuario el nombre, después se pregunta el tipo y tamaño, y se inicializan las variables nuevoA.iskp y nuevoA.null, y nuevoA.sig en -1, con esto se crea un registro nuevo con sus datos listos.

Alta Atributo

Prototipo: void CDiccionario::AltaAtributo();

Esta función es la encargada de verificar que todo esté en condiciones para que se cree un atributo, primero manda a llamar CapturaAt para crear un registro temporal, seguido de esto viene la verificación con la función BuscarAtributo para verificar que no haya un atributo con ese mismo nombre, en caso que no exista usa la función EscribeAtributo(nuevoA) y guarda la dirección en un long dir para finalmente usar la función insertaAtributo para escribir en el archivo.

Buscar Atributo

Prototipo: long CDiccionario::BuscarAtributo(char *atr);

La función long CDiccionario::BuscarAtributo(char *atr) verifica si se encuentra el atributo, en caso de que sí lo encuentre dentro del recorrido while(cab != -1) regresa la dirección en donde se encuentra, en caso contrario regresa un valor de -1, dentro del ciclo hace un string compare de la posición donde se encuentra y el nombre que se le haya pasado.

Lee Atributo

Prototipo: Atributo CDiccionario::leeAtributo(long dir);

Se encarga de extraer el atributo del archivo, esta función llega con una dirección, esa dirección a partir del BuscarAtributo, es usada para moverse dentro del archivo, hacer una copia y regresar esa copia para su posterior modificación o consulta.

Escribe Atributo

Prototipo: `long CDiccionario::EscribeAtributo(Atributo ant);`

Esta función se encarga de registrar físicamente un nuevo atributo en el archivo, donde se le pasa una copia del atributo, la función usa un `fseek` con `SEEK_END` para ir hasta el final del último registro del archivo y en ese espacio de memoria usa un `fwrite` para guardarlo en el archivo.

Inserta Atributo

Prototipo: `void CDiccionario::insertaAtributo(Atributo nvo, long dir);`

Esta función se encarga de enlazar secuencialmente de manera lógica un nuevo atributo verificando tres casos: si en el caso que la cabecera `activa.atr` sea `-1` "Lista vacía", se enlaza directo, el segundo caso ocurre cuando el atributo se encuentra al inicio, o sea que el `strcmp` sea mayor a 0 decidiendo así que va antes, el tercer caso es cuando tiene que hacer una búsqueda apoyándose de un `while` que recorre toda la lista mientras no llegue al final, dentro usa otro `strcmp` para ver en qué lugar se encuentra su inmediato.
newline En esencia la función hace reconexión entre la variable `nvo.sig` para que quede de manera ordenada, todos los `EscribeAtributo` siempre hacen el `write` al final por tanto lo único que cambia esta función son las direcciones a las cuales van apuntando de manera ordenada.

Consultar Atributo

Prototipo: `void CDiccionario::consultarAtributo();`

Esta función es la encargada de imprimir en consola los atributos que se encuentran dentro de nuestro archivo. Para ello, hace uso de `activa.atr` para ver dónde inicia la cabecera del atributo; subsecuentemente hace uso de un ciclo `while` (`cab != -1`), el cual va imprimiendo cada atributo con la dirección de sus componentes.

Baja Atributo

Prototipo: `void CDiccionario::BajaAtributo();`

Su única función es preguntar al usuario por el nombre del atributo, este nombre se lo pasa a la función `eliminaAtributo` en caso de que no exista muestra en consola un aviso al usuario, por el contrario si existe, se hace la desconexión.

Elimina Atributo

Prototipo: `long CDiccionario::eliminaAtributo(char cad[]);`

Esta función empieza verificando la posición desde `activa.atr`, para después verificar si el lugar actual donde está parado es el atributo a borrar, en caso que no hace un recorrido donde hace uso de `while & strcmp`, para encontrar dónde está, usa una variable anterior para poder hacer la desconexión del atributo, no borra nada dentro del archivo solo desconecta y conecta con su inmediato siguiente, lo actualiza en el archivo con la función `reescribeAtributo`.

Reescribe Atributo

Prototipo: `void CDiccionario::reescribeAtributo(long dir, Atributo atr);`

Esta función se encarga de actualizar los datos de un registro ya existente en el archivo binario. Recibe una dirección y una estructura con la información modificada, usa `fseek` con `SEEK_SET` para posicionarse exactamente en la dirección proporcionada, y `fwrite` para sobrescribir el bloque de datos. Es indispensable para actualizar los apuntadores lógicos durante operaciones de inserción y baja.

Modifica Atributo

Prototipo: `void CDiccionario::modificaAtributo();`

Esta función permite actualizar los datos de un registro. Tras verificar su existencia y capturar la nueva información, hace uso de una estrategia segura para mantener el orden alfabético de la lista: elimina lógicamente el registro antiguo con `eliminaAtributo` y después guarda y reubica el atributo actualizado utilizando `reescribeAtributo` e `insertaAtributo`.

4 Diagramas

5 Archivo

6 Clave primaria

7 Conclusiones

Referencias
